

[DOC] Nube

Documentación Técnica · AutoDocs AI · 16/07/2026

11	15	4	3
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 70/100

Documentación del Proyecto

Generado automáticamente por **AutoDocs AI** — 2026-07-16 04:43 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 11 archivos fuente, 15 funciones, 4 clases y 3 endpoints de API documentados. El score de calidad del código es **70/100**.

Stack Tecnológico

Lenguaje	Cantidad
Python	11 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	11
Funciones	15
Clases	4
Endpoints API	3
Score de calidad	70/100
Funciones documentadas	0 / 15
Clases documentadas	0 / 4

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/  
  ■■■ (raíz)/  
  ■■■ controllers/  
  ■■■ entorno/  
  ■■■ models/  
  ■■■ repository/  
  ■■■ services/
```

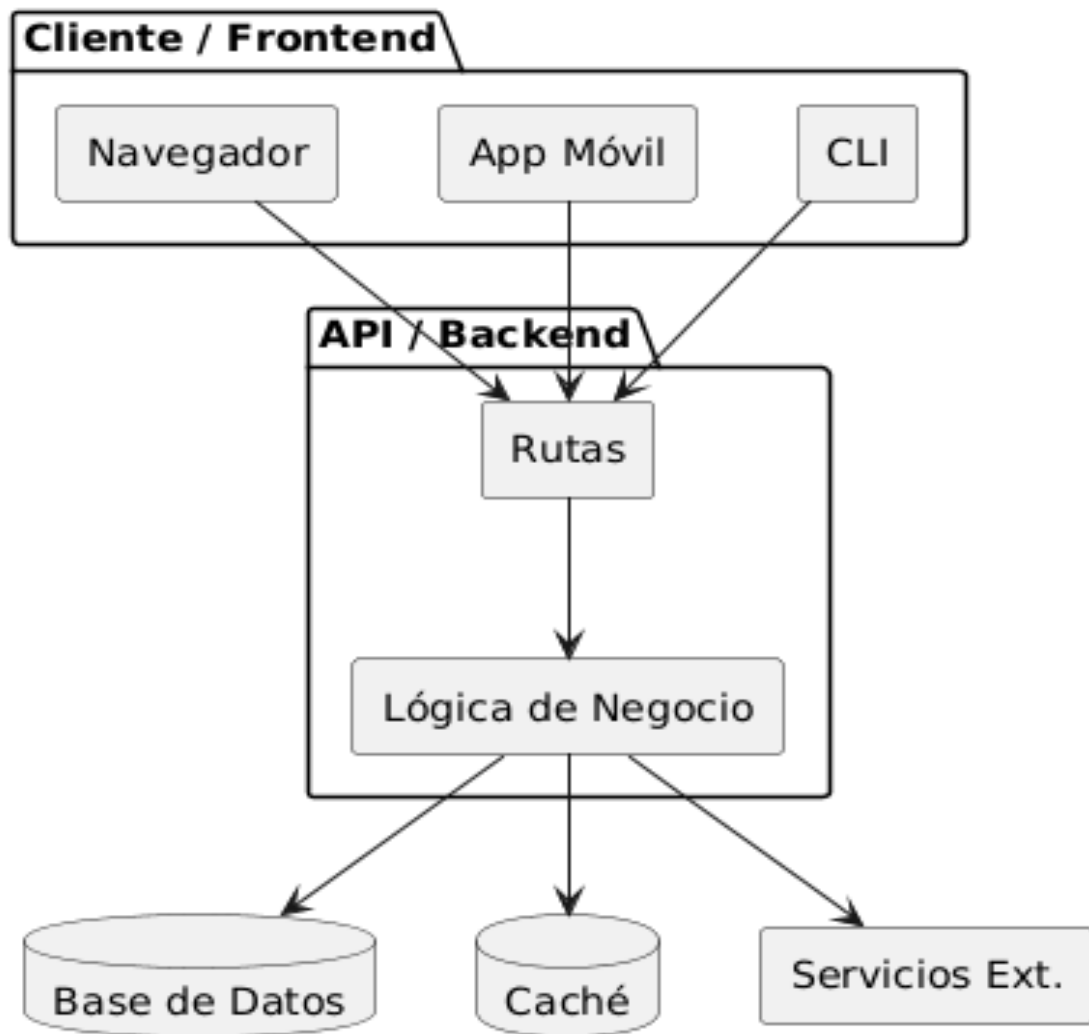
2.2 Distribución por Capas

- (raíz)/ — 8 archivos [.env(1), .example(1), .gitignore(1), .py(2), .md(1), .txt(2)]
- controllers/ — 3 archivos [.py(3)]
- entorno/ — 3 archivos [.py(3)]
- models/ — 1 archivos [.py(1)]
- repository/ — 1 archivos [.py(1)]
- services/ — 1 archivos [.py(1)]

2.3 Distribución de Lenguajes

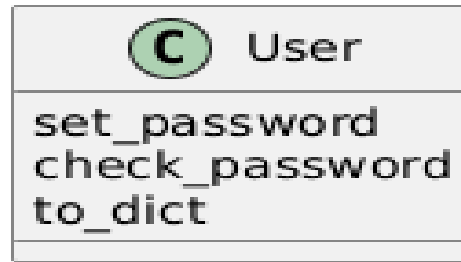
- Python: 11 archivos (100%)

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso

2.6 Diagrama de Clases



■ ■ **Clases con información incompleta:** Las siguientes clases aparecen sin métodos ni atributos documentados: `Config`. Para que el diagrama se visualice correctamente, agrega métodos, propiedades o docstrings a estas clases en tu código fuente y regenera la documentación.

2.7 Patrones Detectados

- **API REST:** 3 endpoints detectados (Flask)

2.8 Insights de Arquitectura (IA)

Insights Arquitectónicos

- **Patrones arquitectónicos probables:** Monolito, debido a la cantidad limitada de endpoints y la simplicidad del proyecto.
- **Buenas prácticas detectadas:** Uso de clases y funciones para organizar el código, lo que sugiere una estructura modular.
- **Áreas de mejora sugeridas:** Aumentar la separación de concerns y considerar la implementación de un patrón de diseño como MVC para mejorar la mantenibilidad.
- **Complejidad del proyecto:** Baja, con solo 15 funciones y 4 clases.
- **Recomendaciones de escalabilidad:** Considerar la implementación de microservicios si el proyecto crece en complejidad o tamaño, y utilizar herramientas de automatización de despliegue para mejorar la eficiencia.

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: 3

Módulo: `controllers\AuthController.py`

[POST] `POST /register`

- **Framework:** Flask
- **Archivo:** `controllers\AuthController.py`

Descripción del Endpoint `/register`

1. Propósito del endpoint:

Este endpoint se utiliza para crear una nueva cuenta de usuario en un sistema o aplicación. En otras palabras, es como llenar un formulario para registrarte en un sitio web o aplicación.

2. Funcionalidad:

Cuando se utiliza este endpoint, se envían los datos del nuevo usuario (como nombre, correo electrónico y contraseña) al servidor. Luego, el servidor verifica si los datos son correctos y si el usuario no ya existe. Si todo está en orden, se crea una nueva cuenta de usuario y se almacena en la base de datos.

3. Información necesaria:

Para que este endpoint funcione, se necesita enviar la siguiente información:

- * Nombre de usuario
- * Correo electrónico
- * Contraseña
- * Otra información adicional que el sistema requiera (como dirección, teléfono, etc.)

Imagina que estás llenando un formulario de registro en un sitio web. Necesitas proporcionar esta información para que el sistema pueda crear tu cuenta.

4. Resultado:

Si todo sale bien, el endpoint devuelve un mensaje que indica que la cuenta se ha creado con éxito. También puede devolver un código de confirmación o un enlace para que el usuario pueda activar su cuenta. Si hay algún error, el endpoint devuelve un mensaje de error que indica qué salió mal.

**5. Uso en la vida

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud
- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado
- `500` — Error interno del servidor

[GET] `GET /register/<int:id>`

- **Framework:** Flask
- **Archivo:** `controllers\AuthController.py`

Descripción del Endpoint de API

1. Propósito del endpoint:

Este endpoint se utiliza para obtener información sobre un usuario que se ha registrado en un sistema. Piensa en ello como una forma de verificar si alguien ya se ha inscrito en una plataforma o servicio en línea.

2. Funcionalidad:

Cuando se utiliza este endpoint, lo que hace es buscar en la base de datos del sistema para encontrar la información de un usuario específico. La forma en que identifica a este usuario es a través de un número único llamado "id". Es como buscar a alguien en una lista de asistentes a un evento, pero en lugar de buscar por nombre, se busca por un código único.

3. Información necesaria:

Para que este endpoint funcione, solo necesita una cosa: el "id" del usuario que deseas buscar. Este "id" es como una clave que abre la puerta a la información del usuario en el sistema. No se necesita nada más, solo ese número.

4. Resultado:

Una vez que el endpoint encuentra la información del usuario, la devuelve. Esto significa que si el "id" es correcto y el usuario existe en el sistema, el endpoint te dará acceso a los datos de ese usuario. Es como recibir un informe con todos los detalles del usuario.

5. Uso en la vida real:

En la vida real, este tipo de endpoint es

Parámetros esperados:

- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
- [400](#) — Solicitud incorrecta
- [401](#) — No autorizado
- [404](#) — Recurso no encontrado
- [500](#) — Error interno del servidor

[POST] [POST /login](#)

- **Framework:** Flask
- **Archivo:** [controllers\AuthController.py](#)

Descripción del Endpoint de Inicio de Sesión

1. Propósito del endpoint:

Este endpoint se utiliza para permitir a los usuarios acceder a un sistema o aplicación de forma segura. Piensa en ello como la puerta de entrada a un sitio web o aplicación que requiere que te identifiques antes de poder usarlo.

2. Funcionalidad:

Cuando un usuario intenta acceder a un sistema, este endpoint se encarga de verificar si la información de inicio de sesión (como el nombre de usuario y la contraseña) es correcta. Es como un portero que verifica tu identidad antes de dejar que entres a un edificio.

3. Información necesaria:

Para que este endpoint funcione, necesita que el usuario proporcione su nombre de usuario y contraseña. Esto se hace a través de una solicitud de inicio de sesión que se envía al servidor. Piensa en ello como llenar un formulario con tus datos de acceso.

4. Resultado:

Si la información de inicio de sesión es correcta, el endpoint devuelve un resultado que indica que el acceso ha sido concedido. Esto puede incluir un token de acceso o una sesión activa que permite al usuario usar el sistema o aplicación. Si la información es incorrecta, el endpoint devuelve un error que indica que el acceso ha sido denegado.

5. Uso en la vida real:

En la vida real

Parámetros esperados:

- **body** (JSON, requerido): Cuerpo de la solicitud
- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado
- 404 — Recurso no encontrado
- 500 — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 15

- [OK] Documentadas: 0 (0%)
- [X] Sin documentar: 15 (100%)

Visualización de documentación:

[■■] 0% documentado

Funciones que requieren documentación:

- `create_app()` en `app.py`
- `register()` en `controllers\AuthController.py`
- `get_user_by_id(id)` en `controllers\AuthController.py`
- `login()` en `controllers\AuthController.py`
- `home()` en `controllers\controller.py`
- ... y 10 más

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200
```

```
def test_post_creates_resource(self, client):
    data = {'name': 'test'}
    response = client.post('/api/endpoint', json=data)
    assert response.status_code == 201

def test_unauthorized_returns_401(self, client):
    response = client.get('/api/protected')
    assert response.status_code == 401
```

Escenarios de prueba recomendados:

Escenario	Tipo	Prioridad
POST /register — respuesta 200	Integración	Alta
POST /register — sin auth (401)	Seguridad	Alta
GET /register/<int:id> — respuesta 200	Integración	Alta
GET /register/<int:id> — sin auth (401)	Seguridad	Alta
POST /login — respuesta 200	Integración	Alta
POST /login — sin auth (401)	Seguridad	Alta

4. Documentación de Clases

Total: 4 clases

Config

- **Archivo:** entorno\config.py
- **Línea:** 6
- **Hereda de:** object
- **Métodos:** 0
- **Documentada:** [X] No

Introducción a la Clase Config

La clase Config es una parte fundamental en nuestro proyecto de Python. Su propósito es simple: actúa como un **almacén de configuración** para nuestro sistema. Piensa en ella como un cajón donde guardamos todas las opciones y ajustes que nuestro sistema necesita para funcionar correctamente.

Qué representa la Clase Config

Imagina que estás configurando un nuevo teléfono. Necesitas decidir qué idioma quieres que tenga, qué hora es, y qué redes de wifi quieres guardar. La clase Config es como el menú de configuración de ese teléfono, pero para nuestro sistema. Almacena todos los ajustes y opciones que nuestro sistema necesita para funcionar de la manera correcta.

Funcionalidades Principales

La clase Config puede hacer varias cosas importantes:

- **Guardar y recuperar configuraciones:** Puedes pensar en esto como guardar tus ajustes favoritos en un juego o aplicación. La clase Config te permite guardar y luego recuperar esas configuraciones cuando las necesitas.
- **Proporcionar acceso a opciones del sistema:** Es como tener un panel de control donde puedes cambiar cómo funciona tu sistema. La clase Config te da acceso a esas opciones de manera sencilla.

Uso en la Vida Real

En la vida real, la clase Config se usa en muchos lugares. Por ejemplo:

- **Aplicaciones de redes sociales:** Cuando config

User

- **Archivo:** `models\user.py`
- **Línea:** 4
- **Hereda de:** `db.Model`
- **Métodos:** 3
- **Documentada:** [X] No

Descripción de la Clase "User"

1. Propósito de la clase:

La clase "User" es como un archivo personal que almacena información sobre cada persona que utiliza un sistema o aplicación. Su propósito es mantener y gestionar los datos de los usuarios de manera segura y organizada.

2. Representación en el sistema:

Imagina un sistema como un hotel. La clase "User" sería como la recepción del hotel, donde se almacenan los datos de cada huésped, como su nombre, dirección, y otros detalles importantes. Al igual que la recepción del hotel, la clase "User" ayuda a identificar y gestionar a cada persona que utiliza el sistema.

3. Funcionalidades principales:

La clase "User" puede hacer varias cosas importantes:

- * **Establecer contraseñas:** Es como crear una combinación segura para una caja fuerte. La clase "User" ayuda a crear y proteger las contraseñas de los usuarios.
- * **Verificar contraseñas:** Es como comprobar si la combinación de la caja fuerte es correcta. La clase "User" verifica si la contraseña ingresada es la correcta.
- * **Convertir datos a un formato útil:** Es como tomar la información de un archivo y convertirla en un formato que sea fácil de leer y entender. La clase "User" puede tomar

Métodos:

- `set_password()`
 - `check_password()`
 - `to_dict()`
-

UserRepository

- **Archivo:** `repository\userRepository.py`
- **Línea:** 4
- **Hereda de:** `object`
- **Métodos:** 4
- **Documentada:** [X] No

Descripción de la clase UserRepository

1. Propósito de la clase:

La clase UserRepository es como un gestor de información de usuarios. Su función principal es almacenar y recuperar información sobre los usuarios de un sistema. Piensa en ella como un archivo donde se guardan todos los datos de los usuarios.

2. Representación en el sistema:

Imagina un sistema de biblioteca donde se almacenan libros. En este caso, la clase UserRepository sería como el catálogo de la biblioteca, donde se guardan todos los registros de los libros (o usuarios, en este caso). Esta clase ayuda a organizar y encontrar la información de los usuarios de manera eficiente.

3. Funcionalidades principales:

La clase UserRepository puede hacer varias cosas:

- **Crear nuevos usuarios:** Es como agregar un nuevo libro al catálogo de la biblioteca.
- **Encontrar un usuario por su ID:** Es como buscar un libro específico en el catálogo usando su código de barras.
- **Encontrar un usuario por su nombre:** Es como buscar libros de un autor específico en el catálogo.
- **Encontrar un usuario por su correo electrónico:** Es como buscar un libro por su título en el catálogo.

4. Uso en la vida real:

En la vida real, esta clase se usa en sistemas que necesitan gestionar información de usuarios, como:

- Sit

Métodos:

- `create()`
- `find_user_by_id()`
- `find_by_user()`
- `find_by_email()`

`authService`

- **Archivo:** `services\authService.py`
- **Línea:** 6
- **Hereda de:** object
- **Métodos:** 3
- **Documentada:** [X] No

Métodos:

- `register()`
 - `find_by_id()`
 - `login()`
-

5. Documentación de Funciones

Total: **15 funciones**

Módulo: `app.py`

`create_app`

- **Parámetros:** —
- **Async:** —
- **Complejidad:** [LOW] 1
- **Documentada:** [X]

La función "create_app" es como un constructor de una aplicación. Sirve para crear una aplicación básica en Python.

Necesita algunos datos para funcionar, como la configuración y los componentes de la aplicación.

Produce una aplicación lista para usarse.

Se utiliza en la vida real para crear aplicaciones web, móviles o de escritorio.

Es importante porque permite crear aplicaciones personalizadas y eficientes.

Módulo: `controllers\AuthController.py`

`register`

- **Parámetros:** —

- Async: —
- Complejidad: [LOW] 4
- Documentada: [X]

La función "register" permite a los usuarios crear una cuenta en un sistema. Para funcionar, necesita información básica como nombre, correo electrónico y contraseña. Una vez proporcionados estos datos, la función crea una nueva cuenta y almacena la información de manera segura. En la vida real, se utiliza para registrarse en sitios web, aplicaciones o plataformas en línea. Es importante porque permite a los usuarios acceder a servicios y funciones exclusivas, manteniendo su información privada y segura. En el sistema, esta función es fundamental para gestionar y autenticar a los usuarios de manera efectiva.

`get_user_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

La función "get_user_by_id" busca un usuario en el sistema según su identificador único. Para funcionar, necesita saber la identificación del usuario que se busca. Al encontrar al usuario, devuelve información sobre él. En la vida real, se usa para recuperar datos de un usuario específico, como en un sistema de gestión de cuentas. Es importante porque permite al sistema identificar y recuperar información de usuarios de manera precisa y eficiente.

`login`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 5
- Documentada: [X]

La función "login" permite a los usuarios acceder a un sistema o plataforma de manera segura.

Necesita saber el nombre de usuario y la contraseña para funcionar.

Una vez que se ingresa esta información, la función verifica si es correcta y, si lo es, permite al usuario acceder al sistema.

En la vida real, se utiliza para acceder a cuentas de correo electrónico, redes sociales o aplicaciones bancarias.

Es fundamental para la seguridad y el control de acceso en cualquier sistema, ya que evita que personas no autorizadas accedan a información confidencial.

Módulo: `controllers\controller.py`

`home`

- Parámetros: —

- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "home" es como una puerta de entrada a un sistema. Su propósito es mostrar la página principal de una aplicación.

Necesita saber qué archivo debe abrir para mostrar la información correcta.

Produce una página web con información útil para el usuario.

En la vida real, se usa en sitios web para mostrar el contenido principal, como el inicio de una tienda en línea o un portal de noticias.

Es importante porque da la primera impresión del sistema y ayuda a los usuarios a encontrar lo que buscan.

Módulo: `models\user.py`

`set_password`

- Parámetros: `self, password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`check_password`

- Parámetros: `self, password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`to_dict`

- Parámetros: `self`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

Módulo: `repository\userRepository.py`

`create`

- Parámetros: `username, email, password`
- Async: —

- Complejidad: [LOW] 1
- Documentada: [X]

`find_user_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`find_by_user`

- Parámetros: `username`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`find_by_email`

- Parámetros: `email`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

Módulo: `services\authService.py`

`register`

- Parámetros: `username`, `email`, `password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`find_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`login`

- Parámetros: `username`, `password`
- Async: —
- Complejidad: [LOW] 2

- Documentada: [X]
-

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

- User By Id
 - By Id
 - By Email
 - By User
 - App
-

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- Lenguaje principal: Python
- Archivos detectados: 11

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
```

```
COPY . .  
RUN pip install -r requirements.txt  
EXPOSE 8000  
CMD ["python app.py"]
```

```
# Construir imagen  
docker build -t mi-proyecto .  
  
# Ejecutar contenedor  
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General  
APP_ENV=production  
SECRET_KEY=clave-secreta-segura  
DEBUG=False  
  
# Base de datos  
DB_HOST=localhost  
DB_PORT=5432  
DB_NAME=mi_base_datos  
DB_USER=usuario  
DB_PASSWORD=contraseña  
  
# API  
API_VERSION=v1  
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- ☐ Variables de entorno configuradas
- ☐ Base de datos migrada
- ☐ Tests ejecutados y pasando
- ☐ Logs configurados
- ☐ HTTPS habilitado en producción
- ☐ Backups programados
- ☐ Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-07-16

- Versión inicial analizada por AutoDocs AI
 - Documentación técnica generada automáticamente
 - Análisis de calidad del código completado
-

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 70/100 (Bueno [YELLOW])

Distribución de Documentación:

- Funciones documentadas: 0/15 (0%)
- Clases documentadas: 0/4

Análisis de Complejidad:

- Complejidad promedio: 1.6
- Complejidad máxima: 5
- Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

- Agregar docstrings a las 15 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

- **Funciones sin documentar:** 15

- `create_app` en `app.py`
- `register` en `controllers\AuthController.py`
- `get_user_by_id` en `controllers\AuthController.py`
- `login` en `controllers\AuthController.py`
- `home` en `controllers\controller.py`
- ... y 10 más

- **Problemas de naming conventions:** 1

- `authService`: Las clases deben empezar con mayúscula (PascalCase)

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	0	Requieren refactorización urgente
■ Media	4–5	2	Candidatas a simplificación
■ Baja	≤ 3	13	Complejidad aceptable

■ No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.

B) Funciones Sin Documentación

15 de 15 funciones carecen de docstring (100%).

Función	Archivo	Línea	Complejidad
<code>`create_app`</code>	<code>`app.py`</code>	8	1
<code>`register`</code>	<code>`controllers\AuthController.py`</code>	36	4
<code>`get_user_by_id`</code>	<code>`controllers\AuthController.py`</code>	79	2
<code>`login`</code>	<code>`controllers\AuthController.py`</code>	93	5
<code>`home`</code>	<code>`controllers\controller.py`</code>	5	1
<code>`set_password`</code>	<code>`models\user.py`</code>	12	1
<code>`check_password`</code>	<code>`models\user.py`</code>	15	1

`to_dict`	`models\user.py`	18	1
`create`	`repository\userRepository.py`	6	1
`find_user_by_id`	`repository\userRepository.py`	19	1
`find_by_user`	`repository\userRepository.py`	23	1
`find_by_email`	`repository\userRepository.py`	27	1
`register`	`services\authService.py`	8	1
`find_by_id`	`services\authService.py`	13	1
`login`	`services\authService.py`	17	2

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

Se detectaron **3 grupos de funciones con nombres similares** repartidas en distintos archivos:

Patrón: `_by_id / by_id` — aparece en 3 archivos distintos

Función	Archivo	Línea	Complejidad
`get_user_by_id`	`controllers\AuthController.py`	79	2
`find_user_by_id`	`repository\userRepository.py`	19	1
`find_by_id`	`services\authService.py`	13	1

■ **Sugerencia:** Las 3 funciones `get_user_by_id`, `find_user_by_id`, `find_by_id` comparten el patrón `by_id` en distintos módulos. Esto sugiere lógica duplicada. Extrae la lógica común a un servicio o función base y haz que cada módulo la llame con sus parámetros propios.

Patrón: `_register / register` — aparece en 2 archivos distintos

Función	Archivo	Línea	Complejidad
`register`	`controllers\AuthController.py`	36	4
`register`	`services\authService.py`	8	1

■ **Sugerencia:** Las funciones `register` y `register` realizan operaciones similares (`register`) en módulos distintos. Considera crear una función compartida en un módulo utilitario (por ejemplo `utils/register.py` o `helpers/common.py`) que reciba el contexto como parámetro, y llamarla desde ambos archivos (`controllers\AuthController.py`, `services\authService.py`).

Patrón: `_login / login` — aparece en 2 archivos distintos

Función	Archivo	Línea	Complejidad
---------	---------	-------	-------------

`login`	`controllers\AuthController.py`	93	5
`login`	`services\authService.py`	17	2

■ **Sugerencia:** Las funciones `login` y `login` realizan operaciones similares (`login`) en módulos distintos. Considera crear una función compartida en un módulo utilitario (por ejemplo `utils/login.py` o `helpers/common.py`) que reciba el contexto como parámetro, y llamarla desde ambos archivos (`controllers\AuthController.py`, `services\authService.py`).

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`repository\userRepository.py`	4	1.0	0%	■ OK
`controllers\AuthController.py`	3	3.7	0%	■ OK
`models\user.py`	3	1.0	0%	■ OK
`services\authService.py`	3	1.3	0%	■ OK
`app.py`	1	1.0	0%	■ OK
`controllers\controller.py`	1	1.0	0%	■ OK

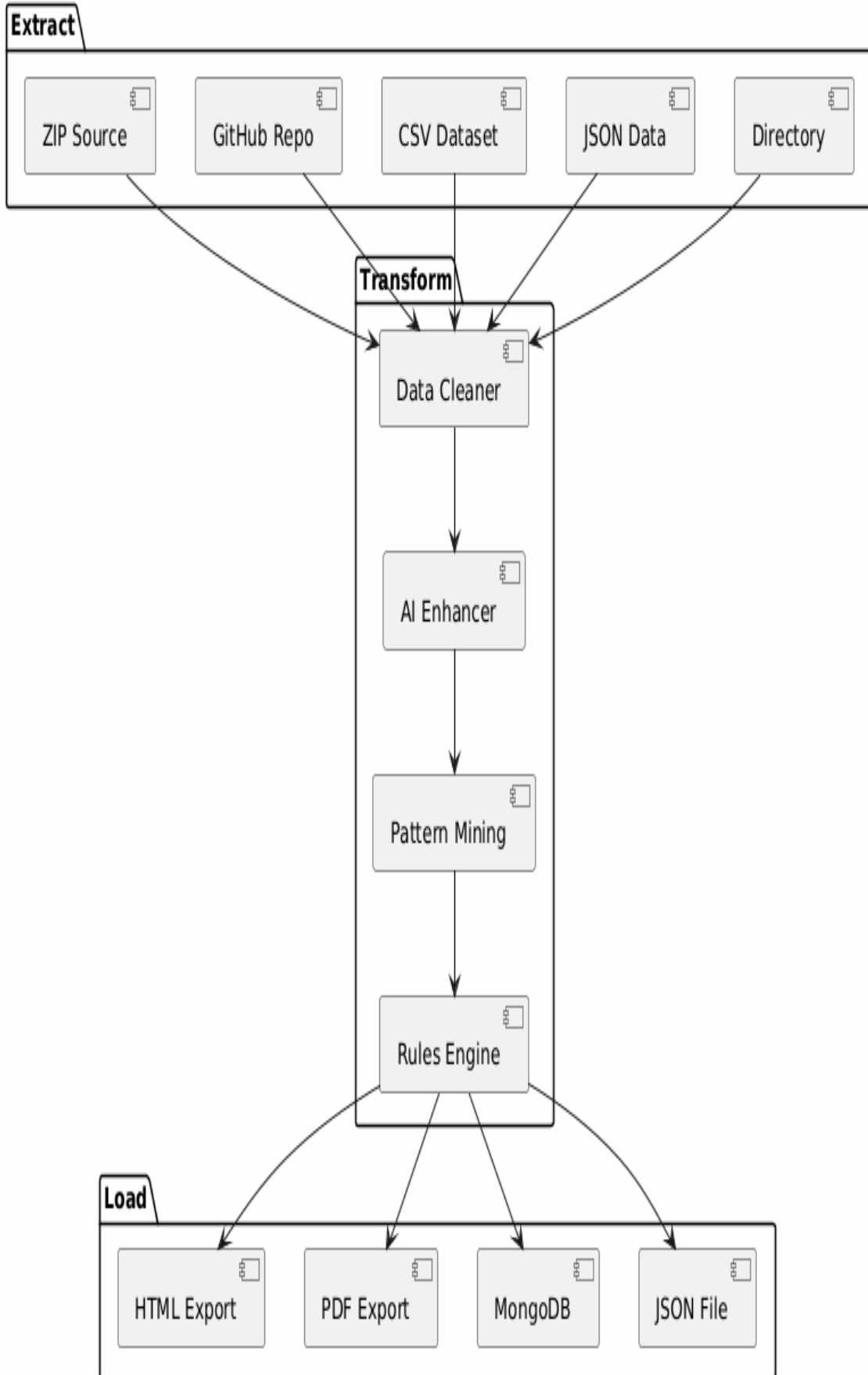
E) Patrones Frecuentes Adicionales (Análisis IA)

- **register/login:** 2 ocurrencias
 - Ejemplos: ['register', 'login']
- **find_by_:** 3 ocurrencias
 - Ejemplos: ['find_user_by_id', 'find_by_user', 'find_by_email']
- **create/get:** 2 ocurrencias
 - Ejemplos: ['create_app', 'get_user_by_id']

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):

Pipeline ETL - AutoDocs AI



Fases del Pipeline:

- **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-07-16 04:43 UTC

Generado automáticamente por AutoDocs AI